B.M.S. COLLEGE OF ENGINEERING

(AUTONOMOUS INSTITUTE, AFFILIATED TO VTU)

BULL TEMPLE ROAD, BASAVANGUDI, BENGALURU - 560019

A Thesis on

VisionFood QAI: AI-Powered Quality Inspection for Food and Beverage Manufacturing

in partial fulfillment of the requirements for the award of the degree

BACHELOR OF ENGINEERING

in

Computer Science & Engineering (Data Science)

by

Shreyas Bairy K S (1BM23CD058)

Sushanth S (1BM23CD063)

Ullas N (1BM23CD067)

Under the Guidance of

Prof. Nayana N Kumar

Project Guide

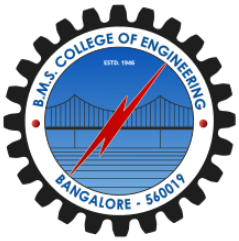
Department of Computer Science and Engineering (Data Science)
B.M.S. College of Engineering

Academic Year 2025–2026

B.M.S. COLLEGE OF ENGINEERING

(AUTONOMOUS INSTITUTE, AFFILIATED TO VTU)

BULL TEMPLE ROAD, BASAVANGUDI, BENGALURU - 560019



DECLARATION

We hereby declare that the project thesis entitled ***VisionFood QAI: AI-Powered Quality Inspection for Food and Beverage Manufacturing*** is a bonafide work carried out by us under the guidance of **Prof. Nayana N Kumar**, Department of Computer Science and Engineering (Data Science), B.M.S. College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Science & Engineering (Data Science) of Visvesvaraya Technological University, Belagavi.

We further declare that, to the best of our knowledge and belief, this thesis has not been submitted either in part or in full to any other university or college for the award of any degree.

Candidate Details

Sl. No.	Student Name	USN	Student Signature
1	Shreyas Bairy K S	1BM23CD058	
2	Sushanth S	1BM23CD063	
3	Ullas N	1BM23CD067	

Date: _____ Place: Bengaluru

B.M.S. COLLEGE OF ENGINEERING

(AUTONOMOUS INSTITUTE, AFFILIATED TO VTU)

BULL TEMPLE ROAD, BASAVANGUDI, BENGALURU - 560019

Department of Computer Science and Engineering (Data Science)

CERTIFICATE

This is to certify that the project thesis entitled ***VisionFood QAI: AI-Powered Quality Inspection for Food and Beverage Manufacturing*** is a bonafide work carried out by the following students for the Capstone Project under the Department of Computer Science and Engineering (Data Science). It is certified that the work reported herein has been carried out under the guidance of **Prof. Nayana N Kumar** and satisfies the academic requirements prescribed for the Bachelor of Engineering degree in Computer Science & Engineering (Data Science).

Candidate Details

Sl. No.	Student Name	USN
1	Shreyas Bairy K S	1BM23CD058
2	Sushanth S	1BM23CD063
3	Ullas N	1BM23CD067

Prof. Nayana N Kumar	**Dr. Shambhavi B R**	**Dr. Bheemsha Arya**
Project Guide	Professor & HOD	Principal

Examiners

Name of Examiner	Signature of Examiner
1.	_____
2.	_____

Acknowledgments

The satisfaction that accompanies the completion of this Capstone Project would be incomplete without acknowledging the support and guidance that made the work possible. We express our sincere gratitude to **Dr. B. S. Ragini Narayan**, Chairperson, Donor Trustee, Member Secretary and Chairperson, BMSET; **Dr. P. Dayananda Pai**, Member Life Trustee, BMSET; and **Dr. Bheemsha Arya**, Principal, B.M.S. College of Engineering, for providing the academic environment and infrastructure required for this project.

We are grateful to **Dr. Shambhavi B R**, Professor and Head of the Department, Computer Science and Engineering (Data Science), and to the project coordinators for their constant support and encouragement. We express our heartfelt thanks to our guide, **Prof. Nayana N Kumar**, for her valuable guidance, timely feedback, and encouragement throughout the project.

We also thank all faculty members of the Department of Computer Science and Engineering (Data Science), our classmates, and everyone who contributed directly or indirectly to the successful progress of this work.

Shreyas Bairy K S (1BM23CD058)
Sushanth S (1BM23CD063)
Ullas N (1BM23CD067)

Abstract

Today's food and beverage manufacturing operates at scales and speeds that challenge manual quality control. Inspectors working on high-speed production lines struggle to maintain consistency and catch small defects-yet every missed flaw can reach consumers, damaging brand reputation and trust. This thesis addresses this critical need by presenting VisionFood QAI, an AI-powered visual inspection system designed specifically for packaged food and beverage products.

Rather than relying on rigid rules or single-defect detection, our approach combines real-world practicality with advanced machine learning. The system learns to identify four common visual problems: improper filling, packaging damage, label misalignment, and surface contamination. Behind the scenes, modern deep learning models-YOLOv8 for detecting defects and EfficientViT-M5 for confirming them, work together to ensure accurate, explainable decisions. To handle uncertainty, we incorporate Monte Carlo Dropout, which quantifies when the model is unsure and flags those cases for human review rather than making risky automatic decisions.

Unlike traditional inspection systems that simply mark products as pass or reject, VisionFood QAIincludes a REMEDY engine that understands product damage realistically. A label that's slightly misaligned might be fixable through relabeling; underfilled bottles can go to a refill station; serious contamination must be quarantined. The system uses SKU-specific product profiles-defining custom inspection rules for bottle_250ml, can_330ml, and pouch_100g formats so the same software pipeline can intelligently inspect very different package types without being told which is which each time.

On the operator side, the system runs FastAPI backend services, logs every inspection for auditing, streams real-time updates, generates shift reports as PDFs, and displays a React dashboard showing live metrics, defect patterns, severity breakdowns, and inspection history. The design prioritizes edge-first deployment using ONNX Runtime, keeping latency below 80 milliseconds per frame so production doesn't slow down.

The literature review identifies significant gaps: beverages are underrepresented in AI quality research, most systems focus narrowly on one defect or package type, and few systems integrate detection with explainability and remediation. We address these gaps through our unified architecture. The project requires a minimum of 400 real annotated images for training, with an ideal target of 1000 or more for production robustness. Our performance targets include reliable defect localization, high sensitivity to real flaws, uncertainty-aware verdicting, and edge latency meeting real manufacturing constraints.

Keywords: Food quality inspection; beverage inspection; deep learning; YOLOv8; explainable AI; edge deployment; operator dashboard.

Contents

Declaration i

Certificate ii

Acknowledgments iii

Abstract iv

Table of Contents v

List of Tables viii

List of Figures ix

List of Abbreviations x

1 Introduction 1

1.1 Background . . . 1
1.2 Objectives . . . 2
1.3 Problem Statement . . . 3
1.4 Project-Specific Design Scope . . . 3
1.5 Summary . . . 4

2 Literature Review 5

2.1 Review Method and Source Selection . . . 5
2.2 CNN and Transfer Learning Approaches . . . 5
2.3 YOLO and Object Detection Methods . . . 6
2.4 Beverage and Packaging-Oriented Machine Vision . . . 7
2.5 IoT and Sensor Fusion Systems . . . 8
2.6 Specialized, Adaptive, and Weakly Supervised Methods . . . 8
2.7 Comparative Summary of Reviewed Papers . . . 9
2.8 Synthesis and Research Gaps . . . 9
2.9 Need for the Proposed System . . . 13
2.10 Chapter Summary . . . 13

3 Methodology 14

3.1 Method Overview . . . 14
3.2 Design Principles . . . 14

3.3 System Architecture . . . 15
3.4 End-to-End Workflow . . . 19
3.5 Project Planning . . . 19
3.6 Requirements . . . 20
3.6.1 Functional Requirements . . . 20
3.6.2 Non-Functional Requirements . . . 20
3.6.3 Software Requirements . . . 20
3.6.4 Development Tooling Requirements . . . 21
3.6.5 Hardware Requirements . . . 21
3.6.6 Data Requirements . . . 21
3.6.7 Operational Requirements . . . 22
3.7 Validation Strategy . . . 22

4 Algorithm, Integration And Testing **23**
4.1 Overview . . . 23
4.2 Algorithm . . . 23
4.2.1 Notation, inputs and outputs . . . 23
4.2.2 Training methodology . . . 24
4.2.3 High-level algorithm . . . 25
4.2.4 Detector and small-object handling . . . 25
4.2.5 Fill-level estimation algorithm . . . 26
4.2.6 Fill-level decision thresholds . . . 26
4.2.7 Cap quality classification . . . 26
4.3 Integration . . . 27
4.3.1 Component interfaces . . . 27
4.3.2 Packaging and deployment . . . 27
4.4 User Interface Implementation . . . 27
4.4.1 Observability and logging . . . 28
4.4.2 Testing hooks and validation points . . . 28
4.5 Cap cropping and small-object detection: detailed explanation . . . 29
4.6 Complexity and runtime notes . . . 29

5 Results and Discussion **30**
5.1 Overview . . . 30
5.2 Experimental Setup . . . 30
5.3 Model Training Results . . . 31
5.4 Inspection Output Validation . . . 31
5.5 System-Level Results . . . 32
5.6 Discussion . . . 32

6 Conclusion and Future Work **34**
6.1 Conclusion 34
6.2 Key Learnings 34
6.3 Comparative Analysis as Future Work 35
6.4 Optimisation Roadmap 35
6.4.1 User Interface Optimisation 35
6.5 Future Work 35
6.6 Sustainable Development Goals Alignment 36
6.7 Closing Summary 36

Appendices **38**
6.8 Dashboard Screenshots 38
6.9 Code Samples 42
6.10 Detailed Matrices 44

References **44**

List of Tables

1.1 Project-specific scope of VisionFood QAI 3

2.1 Comparative summary of papers consulted for VisionFood QAI (Part 1) 10
2.2 Comparative summary of papers consulted for VisionFood QAI (Part 2) 11
2.3 Comparative summary of papers consulted for VisionFood QAI (Part 3) 12

5.1 Cap-inspection training context 30
5.2 Final recorded detector metrics for cap inspection 31

6.1 SDG alignment for VisionFood QAI 37
6.2 Implementation Traceability Matrix 45
6.3 Cap Detection Crop Matrix 45
6.4 Fill-Level Decision Matrix 45

List of Figures

3.1 Primary system architecture showing product routing and inspection flow. . . . 16
3.2 Output combination and evaluation pipeline. 18

6.1 VisionFood QAI dashboard home page 39
6.2 Manual inspection interface with uploaded product image and inference controls 40
6.3 Manual inspection result view showing annotated output and inference details . 41

List of Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
AIoT	Artificial Intelligence of Things
CNN	Convolutional Neural Network
DCAI	Data-Centric Artificial Intelligence
F1	Harmonic mean of precision and recall
HACCP	Hazard Analysis and Critical Control Points
IoT	Internet of Things
mAP	Mean Average Precision
PLC	Programmable Logic Controller
REMEDY	Recommendation, Escalation, Mitigation, Evidence, Diagnosis, and Yield triage engine
RFID	Radio Frequency Identification
SKU	Stock Keeping Unit
XAI	Explainable Artificial Intelligence
YOLO	You Only Look Once

Chapter 1

Introduction

1.1 Background

Food and beverage manufacturing is built on a foundation of quality, safety, and trust. When a customer opens a package expecting a perfect product, they should find exactly that, not a leaking container, dented packaging, misaligned label, or visible contamination. Behind every flawless product on a store shelf lies rigorous quality control. A single oversight can cascade: defective packages leak during transport, compromise food safety, create liability, damage brand reputation, and undermine consumer confidence.

Beverage production lines operate at breathtaking speeds-sometimes hundreds of bottles or cans per minute. In these environments, spotting defects becomes almost superhuman: underfilled bottles, broken seals, dented cans, tilted caps, missing labels, and stains blur past human inspectors. Manual inspection, despite its long history, has fundamental limitations. Inspectors tire during long shifts, their attention drifts, and they miss small defects hidden by reflections on shiny plastic or glass. Yet in many modern facilities, manual visual inspection is still the backbone of quality assurance.

Machine vision has been part of manufacturing for decades. Traditional systems using cameras and rule-based logic can measure fill levels, verify label positions, and detect gross defects. Farhangi et al. showed that careful engineering can achieve 95.6% accuracy on transparent bottles using classical computer vision techniques [1]. However, these rule-based approaches have a hidden cost: they're brittle. Change the bottle design, adjust the label, update the cap color, or switch packaging suppliers and the entire system needs recalibration. For products that evolve regularly, the maintenance burden becomes prohibitive.

Deep learning has opened new possibilities. Convolutional Neural Networks learn visual patterns directly from data, making them more adaptable than hand coded rules. Systems based on ResNet, DenseNet, and similar architectures can now detect food packaging defects with high accuracy [2], [3]. But classification simply marking a product as good or bad isn't enough for operators. They need to know *where* the problem is and *why* it was flagged. YOLO-based object detection answers this need, localizing defects so operators see the exact location of the problem [4].

Despite this progress, a fragmentation persists in research and industry. Beverage inspection forms only a small fraction of ML-based food quality research [5]. Most systems are narrow: inspect only liquid level, or only cap sealing, or only one product family. Real factories need systems that handle *multiple* defect types, work across *different* package formats, provide auditable explanations, and offer practical remediation guidance not just pass/fail verdicts.

VisionFood QAI emerges from this context. It's designed as a complete inspection workflow, not just a model. A camera captures the product, AI localizes candidate defects, uncertainty estimation flags low-confidence decisions, and a REMEDY engine routes products intelligently: some items can be relabeled, others refilled, serious safety issues quarantined. The system uses SKU-specific profiles, so the same software pipeline handles transparent bottles, rigid cans, flexible pouches, and rigid boxes without requiring the model to classify product type in every frame. Rather than a notebook experiment, we've built a production-oriented stack: FastAPI backend, real-time WebSocket updates, database auditing, PDF reports, and a React dashboard that operators actually use.

1.2 Objectives

The primary goal of this project is to design a clear, understandable, and practical deep learning architecture for visual quality inspection in food and beverage manufacturing not just a model, but a deployable system.

Our specific objectives are:

1. To understand and synthesize recent research on deep learning classification, YOLO-based defect detection, IoT-driven quality monitoring, autoencoder-based anomaly detection, weakly supervised approaches, and adaptive learning for industrial inspection.

2. To identify specific limitations in today's food and beverage inspection systems: insufficient beverage research, narrow single-defect focus, lack of localization plus explanation integration, weak remediation support, and scarce public datasets.

3. To design an RGB-camera-based inspection pipeline that can detect and locate four concrete visual problems: improper filling, packaging damage, label misalignment, and surface contamination.

4. To combine YOLOv8 detection with an EfficientViT-M5 classification stage, so that defect locations are supported by a secondary verification and confidence scoring.

5. To implement SKU-level product profiles, allowing configurable inspection of transparent bottles, rigid cans, flexible wrappers, and rigid boxes without forcing the model to re-identify product type constantly.

6. To integrate Monte Carlo Dropout for honest uncertainty estimation and build a REMEDY severity and routing engine so outputs are transparent, auditable, and actionable for plant operators.

7. To define concrete success metrics: precision, recall, F1-score, mAP@0.5, false rejection rates, sub-80-millisecond latency per frame, and complete traceability from image to decision.

1.3 Problem Statement

Manual quality inspection cannot scale with modern manufacturing demands. Human inspectors become fatigued, inconsistent, and miss defects on fast conveyor lines. Yet most industrial systems respond with only pass/reject verdicts offering no path to recovery for fixable problems. The broader research landscape shows that beverage inspection is underexplored, most systems handle only one or two defect types, and few provide the localization, explainability, and operator traceability that plants actually require.

The main challenge we address is the lack of unified, understandable, and operator-ready deep learning architecture for multi-defect visual inspection across different food and beverage package formats. Such a system must:

- Pinpoint defects in images, not just classify them,
- Provide evidence and reasoning that operators can review and trust,
- Support different product formats with configurable rules, not one-size-fits-all logic,
- Offer guidance on remediation (relabel, refill, quarantine, review), not just rejection,
- Generate auditable records so quality decisions are traceable and defensible.

Additionally, the system must respect manufacturing constraints: low-latency edge deployment, reliability on resource limited hardware, and resilience to production-line variability.

Our scope focuses on visual defects visible in RGB images: improper filling, packaging deformation, label placement errors, and surface contamination. We deliberately exclude (for now) chemical spoilage, microbial contamination, and non-visual attributes, though the architecture can integrate additional sensors later.

1.4 Project-Specific Design Scope

This is not an academic model comparison; it's a production-oriented workflow. Every product receives a verdict, evidence, and a permanent record. Table 1.1 captures our scope.

Table 1.1: Project-specific scope of VisionFood QAI

Component	Project-specific decision
Target products	Packaged food and beverage formats, beginning with bottle_250ml, can_330ml, and pouch_100g SKU profiles.
Defect taxonomy	Improper filling, packaging damage, label misalignment, and surface contamination.
Inference design	YOLOv8 detector for localization, EfficientViT-M5 classifier for confirmation, and ONNX Runtime for edge inference.
Decision logic	PASS, FAIL, FAIL with review flag, ESCALATE, and REVIEW based on confidence and uncertainty bands.
REMEDY output	Severity grades S1–S4 and routing actions such as relabel, refill, reject, quarantine, or human inspection.
Operator layer	FastAPI backend, inspection database, WebSocket event stream, React dashboard, KPI cards, Pareto chart, severity chart, and PDF shift report.
Dataset requirement	Minimum 400 real annotated images, ideally 1000 or more, with YOLO-format bounding boxes and a 70:15:15 train-validation-test split.

1.5 Summary

This chapter has outlined why automated visual quality inspection matters in food and beverage manufacturing, and why today's systems often fall short. Manual inspection cannot scale; rule-based systems are inflexible; and most AI systems are too narrow. We've presented VisionFood QAI as a practical answer: a complete pipeline combining deep learning, configurable SKU profiles, uncertainty quantification, REMEDY-based routing, and operator-facing dashboards.

The next chapter surveys the research landscape, synthesizes what works, identifies persistent gaps, and explains how VisionFood QAI addresses them.

Chapter 2

Literature Review

2.1 Review Method and Source Selection

The literature survey was prepared from the research papers stored in the project repository and from the existing project paper draft. The reviewed material covers food quality control, beverage-line inspection, general industrial defect detection, IoT-based food monitoring, packaging inspection, weakly supervised defect detection, and adaptive industrial visual inspection. The review treats sixteen papers as the core technical literature and includes two additional supporting works: the systematic food-quality-control review by Liakos et al. [5] and the partially relevant multiple-instance-learning surface-defect paper by Xu et al. [6]. This allows the survey to remain aligned with the project abstract while still accounting for every paper consulted from the repository.

The papers are grouped thematically instead of chronologically because the project requires a synthesis of methods and gaps rather than a year-wise listing. Each paper was examined against a project module: detection, classification, uncertainty, product-profile configuration, remediation, dashboard traceability, or dataset preparation. The main themes are CNN and transfer learning methods, YOLO and object detection methods, beverage and packaging machine-vision systems, IoT and sensor fusion, and specialized methods such as autoencoders, incremental learning, and weak supervision.

2.2 CNN and Transfer Learning Approaches

CNN-based methods are widely used for image classification in quality inspection because they learn hierarchical visual features directly from images. Banus et al. proposed a deep learning system for quality control of thermoforming food packages and compared ResNet18, ResNet50, VGG19, and DenseNet161 with both pretrained and non-pretrained configurations [2]. Their production-line validation is especially important because the system was tested not only in laboratory settings but also in a real scenario. The best pretrained DenseNet161 configuration achieved very low false positive and false negative rates, demonstrating that deep learning can satisfy strict food-packaging inspection requirements when imaging conditions and defect definitions are carefully controlled.

Subramanian et al. explored optimized ResNet50 for real-time defect detection in manufacturing [3]. The work supports the value of transfer learning, especially where industrial datasets are smaller than general image-recognition datasets. ResNet50 is attractive because residual connections help preserve gradient flow in deeper networks and improve representation learning for cracks, dents, and misalignments. However, the model size and computation

requirements can become challenging for low-cost edge devices unless pruning, quantization, or lightweight backbones are applied.

Chatchapol et al. presented a beverage-specific CNN and image-processing system for liquid filling quality control [7]. The system inspected liquid level and lid sealing while also considering conveyor speed control. It reported 98.89% accuracy for liquid-level detection and 100% accuracy for lid sealing. This paper is directly relevant because it addresses beverage production, but its inspection scope is limited mainly to fill and cap quality rather than the broader set of packaging, label, and contamination defects targeted in VisionFood QAI.

Sanjay proposed a CNN-based food oil quality and usage detection system [8]. The work shows that image-based AI can support food quality assessment beyond package geometry. However, the domain is narrow and does not address industrial production-line localization, traceability, or multi-defect packaging inspection. Xiong et al. proposed a computer-vision-based automated quality-control artifact for food packaging using a data-centric AI approach [9]. Their framework is significant because it evaluates multiple quality factors, including visual and informational aspects, and emphasizes that improving data quality can be more valuable than only changing the model architecture.

Overall, CNN and transfer learning studies show that classification accuracy can be high when the product class and defect taxonomy are well-defined. Their limitation is that classification alone often cannot localize the defective region. For VisionFood QAI, this directly motivates the two-stage design: YOLOv8 first proposes defect regions, and EfficientViT-M5 then checks the cropped region with more class-specific focus. This is more useful to an operator than a pass/fail label because the dashboard can show the defective area, the confirmation class, and the uncertainty attached to the decision.

2.3 YOLO and Object Detection Methods

Object detection is central to industrial inspection because defects must be localized before products can be rejected, reworked, or investigated. YOLO models are especially popular because they perform detection in a single forward pass, making them suitable for real-time production settings.

Kavitha and Mamatha surveyed automated product defect detection using image processing and YOLOv5 for sorting and quality assurance [10]. The paper highlights the transition from manual and rule-based inspection to machine-learning-based detection in manufacturing. A related paper by Kavitha et al. implemented YOLO-based product defect detection for sustainable smart manufacturing and integrated hardware actuation using Arduino-based sorting [4]. This demonstrates an end-to-end inspection-and-sorting concept, but low-cost actuation may not scale directly to high-speed industrial food and beverage lines.

Nguyen et al. proposed an industrial system based on machine vision and YOLOv8, with data multi-threading and PLC integration for real-time operation [11]. The work is important because it moves beyond isolated model training and demonstrates how image acquisition, in-

ference, and actuator communication interact in an industrial pipeline. The reported accuracy above 90% across defect types shows the feasibility of YOLOv8 in production-style environments, although the paper is not specific to food or beverages.

Liu et al. developed an improved YOLOv5 model for automatic food packaging defect detection [12]. The model integrates CBAM attention, feature pyramid and path aggregation networks, and adaptive spatial feature fusion. It was trained on 7400 food-packaging images and reported accuracy 0.96, recall 0.94, and F1-score 0.94. This is the closest prior work to the proposed project because it detects multiple packaging defects in a food-related setting. However, it does not provide operator-facing explainability, remediation guidance, or a broader product-profile mechanism for different package formats.

Gediya discussed a smart manufacturing framework that combines AI image recognition, IoT devices, and edge computing for real-time quality control [13]. Although the paper is more architectural than experimental, it is relevant to VisionFood QAI because it frames quality inspection as part of a larger Industry 4.0 ecosystem. The insight is that detection models must be integrated with data flow, edge processing, production feedback, and continuous improvement mechanisms.

YOLO-based literature confirms that real-time localization is feasible, but most systems stop at detection output. VisionFood QAI treats the detector as only the first decision layer. If YOLOv8 finds no defect, the product follows a fast PASS path. If it finds a defect, the cropped patch is passed to the classifier and uncertainty module before the REMEDY engine selects a corrective path. This project-specific workflow is intended to reduce unnecessary rejection of products that can be relabeled or refilled.

2.4 Beverage and Packaging-Oriented Machine Vision

Beverage inspection presents specific visual challenges. Transparent bottles produce reflections and refractions, fill lines may be faint, labels may wrap around curved surfaces, and caps or seals can be misaligned by only a few pixels. Farhangi et al. addressed cap defects, liquid level, and label placement in liquid bottles using LabVIEW, a CMOS camera, edge detection, and pattern matching [1]. The reported average accuracy of 95.6% shows that classical machine vision can still perform well in constrained bottle-inspection tasks.

The limitation of traditional machine vision is adaptability. A rule tuned for one bottle shape, cap color, or label position may fail after a packaging redesign or SKU change. This motivates the product-profile approach in VisionFood QAI. Instead of hard-coding one global rule set, each SKU can define expected regions of interest, fill bands, label boundaries, severity thresholds, and remediable actions. In the current project plan, YAML profiles are used for bottle_250ml, can_330ml, and pouch_100g, allowing the same inference pipeline to behave differently for different product formats.

The thermoforming study by Banus et al. also offers a practical lesson for packaging inspection: the vision system, imaging configuration, and dataset design are as important as the

network architecture [2]. Their use of production-line imaging and rigorous rejection criteria supports the need for a data-centric pipeline in VisionFood QAI. Similarly, Xiong et al. emphasize that food packaging quality involves both visual conditions and informational correctness [9]. This supports the inclusion of label errors in the target defect taxonomy.

2.5 IoT and Sensor Fusion Systems

Several papers expand quality inspection beyond RGB images by incorporating sensors, cloud systems, and edge analytics. Krishnan et al. proposed an AI-driven intelligent IoT system for real-time food quality monitoring along the supply chain [14]. Their CNN-based system reported 95.2% accuracy, 92.8% precision, 94.5% recall, and 93.6% F1-score, with monitoring results across transport, storage, and processing stages. This shows the value of combining AI models with environmental data such as temperature, humidity, and chemical composition.

Du proposed an optimized online measurement and detection system for intelligent manufacturing using multi-sensor fusion, machine learning, adaptive control, fault prediction, and edge computing [15]. The paper reported 98% surface defect detection accuracy, about 60% improvement in production efficiency, and a 4.85% reduction in scrap rate. Although the domain is general manufacturing, the results support the importance of online inspection and low-latency edge processing.

Cao et al. developed an RFID tag-integrated multi-sensor system with an AIoT cloud platform for food quality analysis [16]. Their system uses semi-passive UHF RFID tags, temperature/humidity/CO2 sensing, cloud visualization, and LSTM-based quality prediction. This approach addresses internal and environmental product quality attributes that cameras may not see. However, cloud-based and sensor-heavy systems can introduce cost, calibration, maintenance, and latency concerns. For VisionFood QAI, RGB vision is the initial focus because it is easier to deploy in a capstone prototype and aligns with available annotation tools. The architecture still leaves room for future sensor values to be passed into REMEDY as additional severity signals.

2.6 Specialized, Adaptive, and Weakly Supervised Methods

Defective samples are often scarce in food and beverage manufacturing because serious defects are rare and may not be consistently captured. Truong and Luong proposed an autoencoder-based, non-destructive approach for detecting defects and contamination in reusable food packaging [17]. Their method trains primarily on clean samples and detects anomalies through reconstruction differences. This reduces dependence on labeled defect images and is highly relevant when collecting examples of rare contamination or cracking is difficult. The trade-off is that anomaly thresholds can be sensitive and may produce false positives if normal product variation is not well modeled.

Ribeiro et al. proposed automatic visual inspection for industrial applications using incremental learning and a multi-view setup for pharmaceutical bottles [18]. Incremental learning

is important because real factories encounter new defects after deployment. Full retraining is expensive and can cause downtime, while learning without forgetting aims to add new defect knowledge without destroying prior performance. This is relevant to VisionFood QAI because food and beverage SKUs, labels, and packaging materials change over time.

Xu et al. introduced a multiple-instance-learning approach for efficient online surface defect detection using image-level labels [6]. This paper is only partially aligned with food and beverage quality inspection, but it is valuable because it addresses the labeling burden. Fine-grained bounding-box annotation is costly, and weak supervision may help bootstrap inspection models when only image-level pass/fail data are available. The limitation is that approximate localization from feature maps may not be precise enough for operator-facing defect evidence.

Liakos et al. conducted a systematic review of machine learning for quality control in the food industry [5]. The review identifies neural networks as a dominant approach and highlights trends such as hyperspectral imaging, sensor fusion, explainable AI, and blockchain-enabled traceability. It also identifies data scarcity, domain coverage bias, regulatory compliance, and integration with legacy systems as persistent challenges. This synthesis strongly supports the motivation for a transparent, auditable inspection architecture.

2.7 Comparative Summary of Reviewed Papers

Table 2.1 summarizes the papers consulted for this project. The comparison focuses on method, domain, contribution, and limitation because these aspects directly inform the design decisions for VisionFood QAI. The final column is written from the viewpoint of the proposed system rather than as a generic criticism of each paper.

2.8 Synthesis and Research Gaps

The reviewed literature shows that automated quality inspection has matured from rule-based computer vision toward deep learning and connected manufacturing systems. However, the findings also show several gaps that motivate VisionFood QAI.

1. **Limited beverage-specific research:** Beverage inspection appears in only a few papers, mainly liquid-level, cap, and label inspection [1], [7]. The broader food QC review also shows that beverages are underrepresented in ML-based food quality control research [5].

2. **Single-defect or narrow-product focus:** Many systems inspect one defect category, one package format, or one production setting. Real plants require inspection across fill level, caps, labels, package damage, and contamination.

3. **Insufficient integration of localization, explanation, remediation, and traceability:** YOLO-based systems localize defects, but few convert detections into explainable visual evidence, severity scoring, recommended action, and auditable operator logs.

Table 2.1: Comparative summary of papers consulted for VisionFood QAI (Part 1)

Study	Domain	Method	Key Contribution	Limitation for This Project
Chatchapol et al. [7]	Beverage filling	CNN with camera module and conveyor speed control	98.89% liquid-level accuracy and 100% lid sealing accuracy in beverage-line context	Focuses on liquid level and sealing; limited multi-defect and XAI support
Kavitha and Mamatha [10]	Manufacturing survey	YOLOv5 and image processing survey	Shows YOLOv5 suitability for fast defect detection and sorting	Survey-level treatment; not food or beverage specific
Sanjay [8]	Food oil quality	CNN image classifier	Demonstrates AI-based visual assessment for food oil quality	Narrow food-oil scope; no localization or operator traceability
Kavitha et al. [4]	Smart manufacturing	YOLO with Arduino sorting	End-to-end detection and physical segregation concept	Low-cost actuation may not scale to industrial food lines
Farhangi et al. [1]	Liquid bottles	LabVIEW machine vision, edge detection, pattern matching	Inspects cap defects, liquid level, and label placement with 95.6% average accuracy	Rule-based approach requires careful tuning for each SKU
Nguyen et al. [11]	Industrial surface inspection	YOLOv8, multi-threading, PLC integration	Demonstrates real-time machine-vision system with above 90% accuracy	Generic industrial products; no food-safety explainability layer

Table 2.2: Comparative summary of papers consulted for VisionFood QAI (Part 2)

Study	**Domain**	**Method**	**Key Contribution**	**Limitation for This Project**
Krishnan et al. [14]	Food supply chain	CNN with IoT sensor monitoring	Reports 95.2% accuracy and robust monitoring across transport, storage, and processing	Sensor-focused monitoring; not localized visual defect inspection
Truong and Luong [17]	Reusable food packaging	Autoencoder anomaly detection and knowledge transfer	Detects contamination and cracks using mostly clean samples	Threshold sensitivity and false positives remain practical risks
Du [15]	Intelligent manufacturing	Multi-sensor fusion, ML, adaptive control, edge computing	Reports 98% defect accuracy and improved production efficiency	General manufacturing; implementation complexity is high
Cao et al. [16]	Food quality monitoring	RFID multi-sensors, AIoT cloud, LSTM	Enables item-level food quality tracking with cloud analytics	Adds hardware, calibration, and cloud-latency concerns
Liakos et al. [5]	Food quality control	PRISMA systematic review	Identifies trends in ML, XAI, sensor fusion, and traceability; highlights beverage gap	Review paper; does not implement an inspection architecture
Ribeiro et al. [18]	Pharma-ceutical bottles	Multi-view visual inspection and incremental learning	Supports adaptation to new defects without full retraining	Pharmaceutical focus; needs adaptation for food and beverage SKUs

Table 2.3: Comparative summary of papers consulted for VisionFood QAI (Part 3)

Study	**Domain**	**Method**	**Key Contribution**	**Limitation for This Project**
Subramanian et al. [3]	Manufacturing defects	Optimized ResNet50 transfer learning	Shows value of residual CNNs for cracks, dents, and misalignments	Classification-heavy; edge deployment may be difficult
Xiong et al. [9]	Food packaging	Data-centric CV framework with DL and traditional CV	Covers visual and informational packaging quality factors	Framework does not provide the proposed REMEDY triage layer
Banus et al. [2]	Thermo-forming food packages	DenseNet, ResNet, VGG comparison	Production-line validation with very low false reject and false accept rates	Focused on thermoforming seals, not broad food and beverage formats
Gediya [13]	Smart manufacturing	AI image recognition, IoT, edge computing	Presents a holistic Industry 4.0 quality-control framework	Lacks detailed food/beverage dataset and defect taxonomy
Liu et al. [12]	Food packaging	Improved YOLOv5 with CBAM, FPN, PANet, ASFF	Closest prior work; 7400 images and 0.96 accuracy for packaging defects	No XAI, remediation guidance, or product-profile-driven routing
Xu et al. [6]	Industrial surface defects	Multiple instance learning with CNN/ ResNet50	Reduces need for fine-grained labels and supports approximate localization	Partially relevant; localization may be insufficient for audits

4. **Edge deployment constraints:** High-performing CNNs and detector models can be computationally expensive. Production inspection requires low-latency inference and reliable operation on edge hardware or near-line GPUs.

5. **Dataset scarcity and annotation burden:** Food and beverage defects are rare, varied, and often proprietary. Autoencoders, weak supervision, and data-centric AI help reduce this burden, but they must be balanced against false positives and audit requirements [6], [9], [17].

2.9 Need for the Proposed System

The literature supports the need for a system that combines the strengths of existing approaches while addressing their operational gaps. CNNs and transfer learning provide strong image understanding, YOLO models provide real-time localization, IoT studies demonstrate the value of traceable quality data, and specialized methods help handle scarce defect samples. VisionFood QAI builds on these directions by proposing a product-profile-driven architecture with configurable SKU rules, YOLOv8 localization, EfficientViT-M5 confirmation, Monte Carlo Dropout uncertainty, REMEDY triage, and a web dashboard for inspection review.

The central design choice is to separate product context from defect localization. Instead of forcing the vision model to infer product type in every frame, the production line or operator selects an SKU profile that defines expected regions, acceptable limits, defect classes, severity rules, and possible corrective actions. This makes the system more practical for transparent bottles, cans, wrappers, and boxes, where visual characteristics differ substantially. The literature review therefore establishes both the feasibility of deep learning based inspection and the need for an integrated, auditable, operator-facing pipeline.

For the capstone implementation, this means that a detected label error on a bottle may be routed to a relabel station if severity is S1 or S2, while visible contamination may be quarantined as S4 because it is a food-safety risk. Similarly, an uncertain low-confidence prediction is not silently accepted or rejected; it is marked for review so that the operator can inspect the image, bounding box, and model confidence. This is the main difference between VisionFood QAI and prior work that ends at a detector output.

2.10 Chapter Summary

This chapter reviewed the research papers consulted for the project and grouped them into CNN and transfer learning methods, YOLO and object detection systems, beverage and packaging machine vision, IoT and sensor fusion, and specialized adaptive methods. The review shows that high accuracy is achievable in controlled settings, but existing work rarely combines multi-defect inspection, beverage relevance, localization, explainability, remediation, traceability, and edge-aware deployment. These gaps form the foundation for the VisionFood QAI architecture proposed in the subsequent stages of the capstone project.

Chapter 3

Methodology

3.1 Method Overview

The methodology for VisionFood QAI is based on a modular inspection pipeline rather than a single end-to-end classifier. That choice was deliberate. In a food and beverage setting, the same camera frame may need to support different checks depending on the product family, the packaging format, and the defect class under review. A rigid monolithic model would blur those distinctions and make the final decision harder to explain. The architecture used in this project instead separates product routing, defect detection, evidence combination, confidence handling, and remediation so that each step can be inspected independently.

At a practical level, the workflow starts with a captured RGB frame, assigns the frame to the appropriate product branch, executes the relevant inspection modules, and then consolidates the resulting evidence into a verdict. The pipeline is therefore not only concerned with whether a defect exists. It is also concerned with what the defect is, where it is located, how confident the system is, and what the next action should be. That structure is important because the intended user is not a data scientist but an operator who needs a clear and traceable answer.

The two figures shown here serve as the main architectural references for this chapter. The first image presents the product routing and inspection path. The second image shows how outputs are combined, scored, and routed into the REMEDY layer for final handling.

3.2 Design Principles

Three design principles guided the methodology.

First, the system is product aware. Food products and beverage products do not fail in the same way, so the inspection logic cannot be identical. A snack wrapper, a rigid can, a carton, and a transparent bottle each expose different visual cues. The pipeline therefore treats product type as a first-class input instead of trying to infer everything from a single detector output.

Second, the system is evidence driven. Every decision should be grounded in visible image evidence rather than only in a scalar score. Bounding boxes, confidence values, uncertainty estimates, and severity outputs are preserved so that the final result can be audited later.

Third, the system is fail-safe rather than optimistic. If a product region is uncertain, if the fill level cannot be observed clearly, or if the label evidence does not match the SKU profile, the pipeline should not silently force a pass. The design favors review, escalation, or rejection when the evidence is incomplete.

3.3 System Architecture

The system architecture is organised into three logical layers: input acquisition, inspection, and decision support. The input layer captures a single camera frame from the production line or development setup. The inspection layer isolates the product, runs the appropriate detectors, and converts raw image findings into defect objects. The decision-support layer uses uncertainty and severity logic to decide whether the item should pass, be escalated, or be routed to corrective handling.

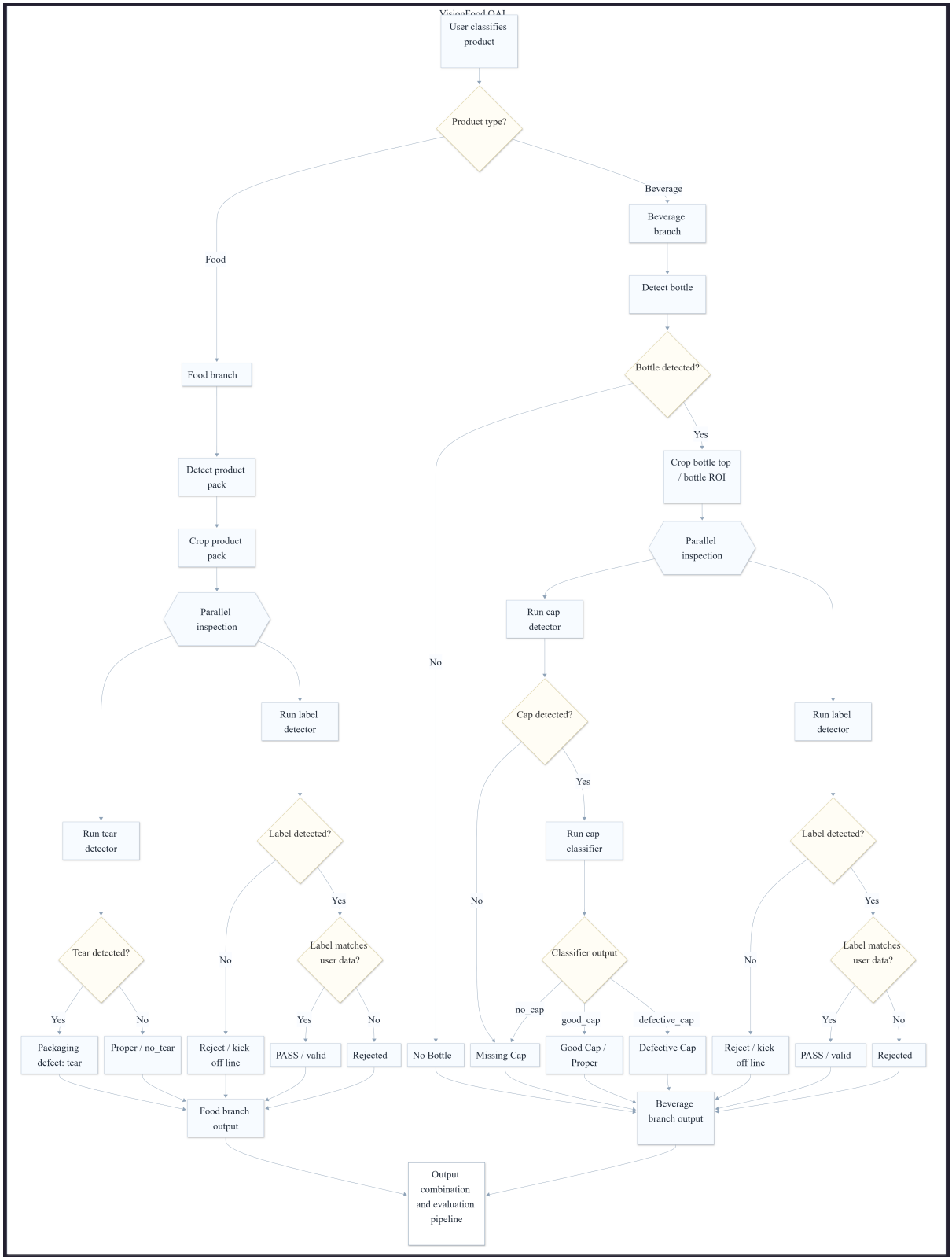


Figure 3.1: Primary system architecture showing product routing and inspection flow.

Figure 3.1 shows the first stage of the architecture in operational terms. The image is not just a diagram of model blocks; it represents the actual inspection route. A frame enters the system, the product context is identified, and the frame is passed to the relevant branch. From there, the branch-specific checks are applied. For food products, the system focuses on packaging integrity and label quality. For beverage products, the system focuses on bottle geometry, cap fitting, fill level, and surface condition.

This separation reduces unnecessary computation and avoids applying irrelevant checks to the wrong product type. A transparent bottle, for example, requires fill-level reasoning that is meaningless for a carton. Similarly, a carton may require corner and crease inspection that does not apply to a rigid can. The architecture therefore keeps the routing logic above the defect logic so that each product category can be inspected according to its own rules.

The first stage also aligns with the SKU profile concept used in the implementation. In practice, the selected SKU tells the system what product family it is handling, which inspection rules are valid, and which defect classes should be emphasized. This makes the workflow configurable without requiring the detector layer to be rewritten for every new package.

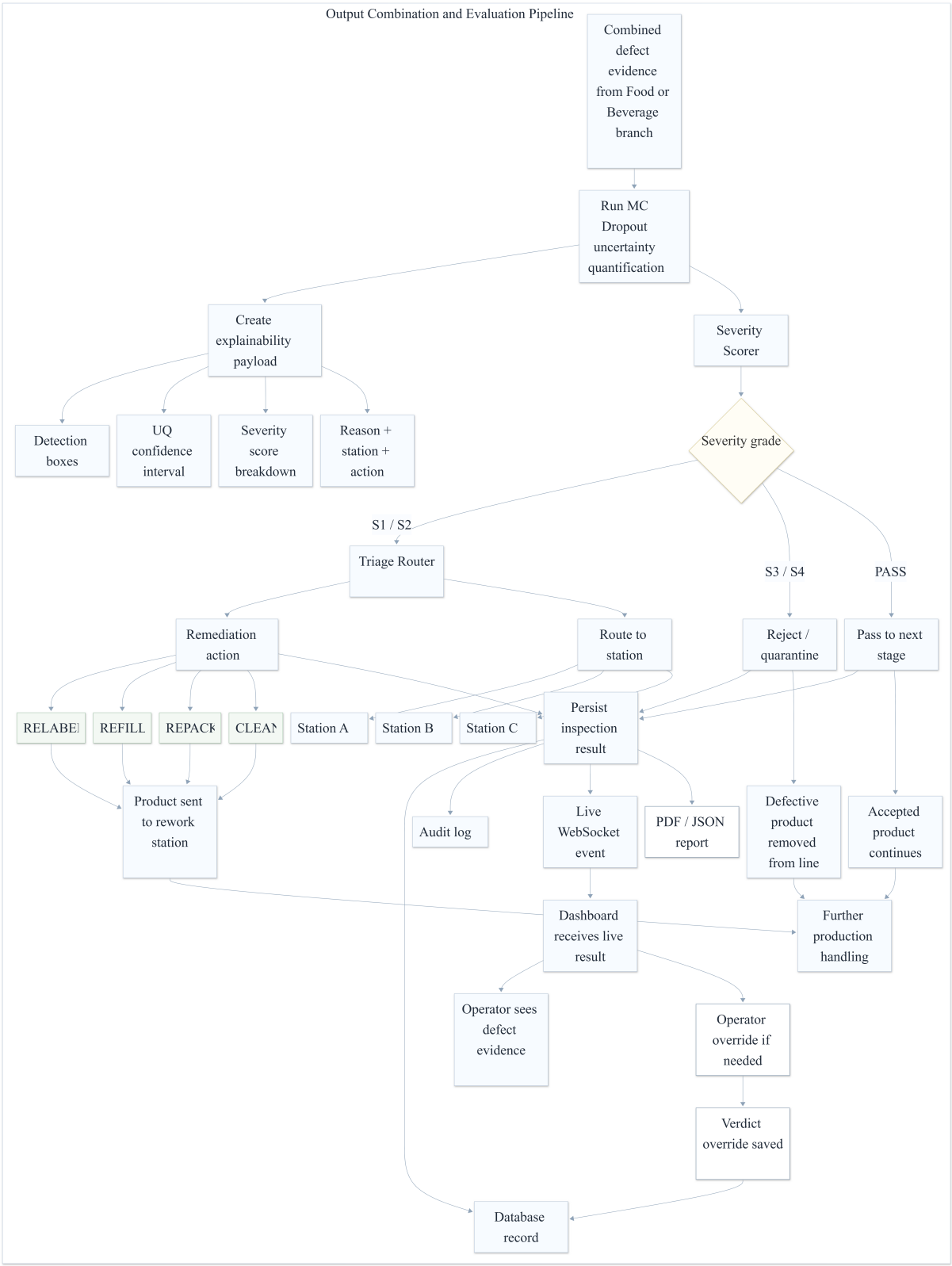


Figure 3.2: Output combination and evaluation pipeline.

Figure 3.2 captures the downstream decision layer. After the inspection modules finish, their outputs are not consumed in isolation. Instead, the system combines the detections, evaluates how stable or uncertain the prediction is, estimates severity, and then selects the response path. That downstream logic is central to the project because two defects with the same visual appearance may require different actions depending on confidence, product profile, and operational context.

This stage also makes the pipeline easier to review. The output includes the defect evidence, the confidence context, the severity grade, and the final route. In other words, the architecture supports both machine decision-making and human review. That is important in quality inspection because operators often need to justify why a product was escalated, held, or rejected.

3.4 End-to-End Workflow

The implementation follows a defined sequence designed to keep the inspection process deterministic and reproducible.

1. A frame is captured from the imaging source and passed into the pipeline.
2. The product context is resolved using the selected SKU profile.
3. The appropriate inspection branch is executed for the product category and subtype.
4. The branch returns defect objects and associated evidence.
5. The system evaluates whether the results are confident enough for automatic handling or whether they require review.
6. The severity and triage logic convert the inspection output into an action such as pass, escalate, rework, or reject.
7. The complete result is stored so it can be audited or displayed in the dashboard.

This workflow reflects the logic of an industrial inspection station. A product is not assessed by a single detector in isolation; rather, the final decision is derived from product identity, defect visibility, confidence, and corrective feasibility. Making those stages explicit improves traceability, supports debugging, and strengthens the justification for the final result during evaluation.

3.5 Project Planning

The project was organised into phases so that the scope could expand in a controlled manner without becoming fragmented. The first phase focused on problem definition and literature review. This stage clarified the relevant failure modes in food and beverage inspection, identified areas where prior work was effective, and isolated the remaining gaps. It also established that the system should support multiple package types rather than a single product configuration.

The next phase focused on architectural decomposition. Instead of organising the work around file names or model artefacts, the implementation was structured around functional inspection tasks: product routing, defect localisation, branch-specific checks, uncertainty estimation, and decision handling. This approach made the system easier to extend because each block could be implemented, tested, and documented independently.

The third phase covered data and profile preparation. In this project, data is not limited to image files. It also includes the SKU profile, the class mapping, the expected product format, and the rules that determine whether a detection is operationally significant. Addressing these dependencies early reduces the risk of developing a model that performs well in isolation but is difficult to use within the inspection pipeline.

The final phase focused on reporting and presentation. The report was written to describe the implemented pipeline rather than a hypothetical alternative, which keeps the methodology aligned with the system architecture presented in the thesis.

3.6 Requirements

The project requirements were derived from the operational needs of a real inspection workflow. They are grouped into functional, non-functional, software, development tooling, hardware, data, and operational requirements so that each aspect of the system can be evaluated independently.

3.6.1 Functional Requirements

The system had to detect the product in the image, inspect the relevant defect types, and produce a final verdict. It also had to support separate food and beverage branches, generate explainable output, preserve the inspection evidence, and retain the result for later review in the dashboard or database.

In addition, the pipeline had to support product-aware rules. That means the same image could lead to different outcomes depending on the SKU profile, the expected defect classes, and the allowable limits for that product.

3.6.2 Non-Functional Requirements

The system had to be modular, accurate, and fast enough for inspection use. It also had to remain interpretable for operators, tolerant of variable lighting and packaging appearance, and structured so that future updates could be added without replacing the full pipeline.

Traceability was another important non-functional requirement. The output should support later review by keeping the defect class, score, and decision context together instead of collapsing everything into a single label.

3.6.3 Software Requirements

The implementation required Python-based computer vision support, deep-learning inference tooling, image-processing libraries, and a backend capable of storing and serving inspection

results. The report and presentation were prepared in LaTeX, which made it possible to keep the documentation structured and consistent with the thesis format.

The software stack also had to support future extension. For that reason, the methodology assumes clear schema definitions, reusable inspection modules, and a pipeline interface that can evolve without forcing the UI or database layer to change in lockstep.

3.6.4 Development Tooling Requirements

Version control with Git was required so that changes to the code, configuration, and documentation could be tracked systematically across the life of the project. A versioned workflow also provided a reliable way to compare revisions, isolate experimental changes, and preserve stable checkpoints when the implementation evolved.

Roboflow was required for dataset annotation and management. It provided a structured environment for drawing bounding boxes, organising defect classes, and exporting annotations in a consistent format for downstream training and evaluation. This requirement is important because inspection performance depends heavily on annotation quality and class consistency.

3.6.5 Hardware Requirements

The methodology assumes a camera-based imaging setup and a compute device capable of running the inspection pipeline at practical speed. Depending on deployment, this could be a development workstation during experimentation or an edge-style inference device in a production setting.

For beverage inspection, the imaging setup must expose enough contrast for fill-level reasoning. For packaging inspection, the lighting must be stable enough to preserve contours, labels, and visible surface defects.

3.6.6 Data Requirements

The system required annotated product images covering both normal and defective samples. The data also had to represent the package types used in the architecture so that each inspection branch could be evaluated on the correct visual material.

Data quality mattered as much as data quantity. If the annotations are inconsistent, the downstream defect logic becomes unreliable. For that reason, the dataset requirements include consistent class names, visible defect regions, and product formats that match the SKU profiles used in the pipeline.

To support this process, images were prepared and labeled in Roboflow before being exported for training and evaluation. This ensured that the annotation format remained uniform across defect categories and that the resulting dataset could be reused across experiments without reworking labels each time.

3.6.7 Operational Requirements

The output had to support operator review, traceability, and actionability. In practice, that means the system needed to provide more than a pass or fail answer. It needed to show what was detected, where it was detected, how strong the evidence was, and why the final decision was selected.

This requirement is especially important for uncertain or borderline cases. If the evidence is incomplete, the system should communicate that clearly instead of hiding it behind an overly confident verdict.

3.7 Validation Strategy

The methodology was also planned with verification in mind. Each inspection branch should be testable on its own using synthetic or controlled images, and the final pipeline should be checked end to end with sample frames that exercise the routing logic, the evidence combination stage, and the verdict rules.

The validation strategy therefore focuses on three questions: whether the correct branch is selected, whether the correct defect evidence is extracted, and whether the final result is consistent with the configured thresholds and product rules. That approach helps ensure that the pipeline remains stable as additional product categories or defect classes are added later.

Chapter 4

Algorithm, Integration And Testing

4.1 Overview

This chapter describes the implemented inference flow used for bottle inspection and the component-level integration needed to run it end to end. The pipeline combines a YOLO detector for bottles and caps (with a two-pass zoom for small caps), a MobileNetV3-Small classifier for cap quality, and a water-surface detector plus fill-ratio math for fill-level verdicts. The presentation follows a formal, implementation-aware style: inputs and outputs are specified, the core steps are enumerated, key equations are shown, and integration interfaces are defined for testing and deployment.

4.2 Algorithm

4.2.1 Notation, inputs and outputs

Let an input camera frame be $I \in \mathbb{R}^{H\times W\times 3}$. The detector produces a set of candidate bounding boxes $\mathscr{B} = \{b_i\}_{i=1}^{N}$ where $b_i = \{x_1, y_1, x_2, y_2, c_i, cls_i\}$ with confidence c_i and class cls_i. Each downstream module maps a crop $C(b_i) = I[y_1 : y_2, x_1 : x_2]$ to a structured result. Example output schemas used in the pipeline are:

- Detection: $\{b_i, c_i, cls_i\}$
- Cap classification: $\{label_{cap}, p_{cap}\}$ (cap label and confidence)
- Fill-level estimate: $\{f_{fill}, v_{fill}\}$ (fill ratio and verdict)
- Per-bottle result: $\{bottle_bbox, cap_verdict, fill_verdict, latency_ms\}$.

Detailed notation and symbols

For clarity we summarize the symbols and data structures used throughout the algorithm descriptions and equations.

I**:** Input image frame, shape $H \times W \times 3$.

b_i**:** Bounding box for detection i, coordinates in pixel space $b_i = (x_1, y_1, x_2, y_2)$.

c_i**:** Detector confidence for box b_i (scalar in $[0, 1]$).

cls_i**:** Predicted class label for box b_i (categorical index).

$C(b_i)$**:** Image crop extracted from I using box b_i.

$\mathscr{B}$: Set of boxes returned by the detector for a frame.

y_{top}, y_{bot}: Corrected bottle bounds after trimming the detector box.

y_{water}: Vertical coordinate of the detected water surface (bbox center).

f_{fill}: Fill ratio computed from the bottle height and water surface location (0 = empty, 1 = full).

v_{fill}: Fill-level verdict in {underfill, normal, overfill}.

p_{cap}: Cap classifier confidence for the selected cap crop.

$\tau_{under}, \tau_{over}$: Decision thresholds for underfill and overfill.

α: Zoom scale factor used in the second-pass cap search.

r_{cap}: Cap-region ratio (top portion of bottle used for zoom crop).

$T_{det}, T_{cls}, T_{fill}, T_{post}$: Latencies for detector, classifier, fill model, and postprocessing steps.

These symbols are used consistently in pseudocode and equations that describe the implemented pipeline.

4.2.2 Training methodology

Training focused on the three models used in the runtime pipeline: the bottle and cap detector, the cap-quality classifier, and the water-surface detector. Fill-level classification models are not part of the deployed inference path and are therefore omitted here.

Bottle and cap detector (YOLO) The detector is trained to localize bottles and caps from full-frame images. Training uses standard YOLO augmentation defaults and class-weighting for imbalance, with best-checkpoint selection based on validation mAP. The inference pipeline then applies a two-pass zoom to improve cap recall on small objects.

Water-surface detector (YOLO) A dedicated detector is trained to localize the liquid-air interface as a single class `water_surface`. The model outputs a tight bounding box around the visible surface line. During inference, the most plausible surface candidate within each bottle is selected and converted into a fill ratio.

Cap quality classifier (MobileNetV3-Small) The cap classifier is MobileNetV3-Small with a compact head. Training uses letterbox resizing, ImageNet normalization, and heavy augmentation (motion blur, glare simulation, color shift, and random erasing) to reflect conveyor motion and lighting variation. Focal Loss ($\gamma = 2.0$) and class-weighted sampling address class imbalance. The trained weights are saved as a `.pth` checkpoint and used directly during inference.

Data and annotation protocol Annotations follow YOLO-style bounding boxes for bottle, cap, and water-surface classes. Data splits follow a 70:15:15 pattern (train:val:test), and augmentation is used to expand rare defect examples while preserving label integrity.

Validation and selection Validation metrics include mAP for detectors and accuracy, precision, recall, and F1-score for the cap classifier. The best checkpoints are selected based on validation performance and used in the inference pipeline.

4.2.3 High-level algorithm

The per-frame algorithm is summarized in steps and in the pseudocode listing that follows.

```
def inspect_frame(I, det_model, fill_model, cap_cls=None):
  bottles, caps = detect_bottles_and_caps(I, det_model)
  results = []
  for b in bottles:
    cap = select_cap_for_bottle(b, caps)
    cap_verdict, cap_conf = classify_cap_if_available(I, cap,
       cap_cls)
    water = detect_water_surface(I, fill_model, b)
    fill_ratio, fill_verdict = compute_fill_ratio_and_verdict(b,
       water)
    results.append({
      "bottle": b,
      "cap_verdict": cap_verdict,
      "cap_conf": cap_conf,
      "fill_ratio": fill_ratio,
      "fill_verdict": fill_verdict,
    })
  return results
```

Listing 4.1: Per-frame inspection pseudocode

4.2.4 Detector and small-object handling

The detector is YOLO-based and is responsible for proposing bottle and cap regions. Small-object handling for caps uses an explicit two-pass zoom strategy:

1. Pass 1: run the detector on the full frame to obtain bottles and any high-confidence caps.
2. If pass-1 cap confidence is low, crop the top r_{cap} portion of each bottle with padding and resize by a zoom factor α.
3. Pass 2: re-run the detector on the zoomed crop to recover small caps with higher effective resolution.

4. Merge pass-1 and pass-2 caps using IoU and keep the higher-confidence box.
5. Associate the final cap to a bottle by horizontal alignment and minimum distance to the bottle top.

This two-pass approach compensates for small-object limits in single-shot detection and improves cap recall without requiring a heavier detector.

4.2.5 Fill-level estimation algorithm

The implemented fill-level estimator uses a water-surface detector and geometric ratio computation. The goal is to convert a detected liquid-air interface into a fill ratio for each bottle.

Preprocessing The bottle bounding box is corrected for detector padding by trimming a small fraction from the top and bottom. This produces corrected bounds (y_{top}, y_{bot}) that better represent the true bottle interior.

Surface detection A dedicated detector finds the water-surface line as a bounding box. For each bottle, the most plausible surface candidate is selected by horizontal alignment (surface center x within the bottle bounds) and vertical inclusion (surface y within the bottle bounds). If no surface is found, the fill result is left undefined for that bottle.

Fill ratio The water-surface y coordinate is taken as the center of the surface bounding box. The fill ratio is computed by

$$f_{fill} = \frac{y_{bot} - y_{water}}{y_{bot} - y_{top}}. \tag{4.1}$$

This yields $f_{fill} \in [0,1]$ where 0 indicates empty and 1 indicates full, with values in between representing partial fill.

4.2.6 Fill-level decision thresholds

Fill-level decisions are derived directly from the fill ratio. Given thresholds τ_{under} and τ_{over}:

$$v_{fill} = \begin{cases} \text{underfill}, & f_{fill} < \tau_{under} \\ \text{overfill}, & f_{fill} > \tau_{over} \\ \text{normal}, & \text{otherwise} \end{cases} \tag{4.2}$$

If no valid water-surface detection is available for a bottle, the fill-level verdict is left undefined for that instance.

4.2.7 Cap quality classification

Cap quality is verified by a MobileNetV3-Small classifier applied to a cropped cap region. The crop is letterbox-resized to preserve aspect ratio and normalized using ImageNet mean and

standard deviation. The classifier outputs a label from `good_cap`, `defective_cap`, or `no_cap` along with a confidence score p_{cap}. The verdict is mapped as follows:

- `good_cap` → **Good Cap**
- `defective_cap` → **Defective Cap**
- `no_cap` → **Missing Cap**

If the cap classifier is not loaded, the pipeline defaults to **Cap Present** whenever a cap detection exists and **Missing Cap** when it does not.

4.3 Integration

4.3.1 Component interfaces

Each module exposes a minimal API for integration and testing. Examples (Python-like signatures):

```
def detector.detect(frame: np.ndarray) -> dict   # bottles, caps,
    metadata
def classify_cap(classifier, device, crop: np.ndarray) ->
    tuple[str, float]
def compute_fill_ratio(bottle_bbox: list[int], water_bbox:
    list[int]) -> float
def fill_verdict(fill_ratio: float, under: float, over: float) ->
    str
def full_inspect(detector, fill_model, classifier, cls_device,
    frame) -> dict
```

The per-bottle result is serializable to JSON and contains the structured evidence used by downstream storage and UI layers.

4.3.2 Packaging and deployment

The runtime loads YOLO detector weights for bottles and caps, YOLO weights for the water-surface detector, and a MobileNetV3-Small classifier checkpoint for cap quality. The pipeline can run in batch mode on image folders or in webcam mode, producing annotated images and per-bottle JSON summaries. The inspection API endpoint accepts base64 images and returns a structured result for storage and dashboard display.

4.4 User Interface Implementation

The implemented dashboard provides an operator-facing interface around the inspection pipeline. It is built as a React and Vite application with TypeScript types aligned to the back-end inspection schema. The interface includes an operations overview, live monitor, manual

inspection panel, inspection history, escalation queue, product management, report generation, and settings workflows.

The manual inspection panel allows an operator to upload an image, enter SKU and product information, run inspection, and view the returned verdict. The result view displays verdict badges, severity information, latency, annotated image overlays, detection details, REMEDY action, and uncertainty information. This supports manual validation because the user can inspect both the decision and the evidence in the same screen.

The live monitor and run setup components support production-line operation. The run setup banner shows whether a production run is active, lets an operator start a run, and allows a supervisor or administrator to end or abort a run. The active-run view displays SKU, product name when available, product sub-type badge, container contents badge, inspection count, defect count, and elapsed duration. The history and detail views support traceability by displaying stored inspection results, annotated images, detections, severity assessment, remediation action, and label or QR information when available.

4.4.1 Observability and logging

Key runtime metrics for monitoring and testing include per-frame latency, bottle count, cap-detection pass-1 and pass-2 counts, cap verdict distribution, and fill-ratio statistics. These metrics support post-deployment calibration and automated test assertions.

4.4.2 Testing hooks and validation points

Testing follows the modular structure of the pipeline so that each stage can be verified in isolation before end-to-end runs. The scripts are designed to accept a single image, a folder of images, or a live webcam stream and produce both annotated outputs and structured per-bottle summaries. This makes regression checks repeatable and keeps failures local to the responsible module.

Key testing scripts and their roles are:

- `detect_cap_presence.py`: quick binary check for cap presence using the bottle+cap detector; validates class mapping and detector thresholds.
- `only_yolo.py`: detector-only pipeline with the two-pass zoom logic; stresses small-cap recovery and pass-1/pass-2 merging.
- `yoloWithClassifier.py`: integrates the MobileNetV3 cap classifier with detection to validate crop quality, preprocessing, and classifier confidence.
- `yoloWithFillLevel.py`: full bottle pipeline that adds water-surface detection and fill-ratio computation; used to verify fill thresholds and reporting.

Validation uses synthetic frames with known geometry for detector checks, controlled fill-level images to verify f_{fill} and thresholding, and small-object stress sets to validate the crop-

and-zoom logic. The resulting annotated images and per-bottle JSON summaries are used as regression fixtures in the dedicated Testing chapter.

4.5 Cap cropping and small-object detection: detailed explanation

Cap detection often fails when caps occupy only a few pixels in the full frame. To mitigate this, the implemented pipeline uses a crop-and-zoom refinement with the following steps:

1. Base detection: run YOLO on the full frame to obtain bottle and cap candidates.
2. Skip check: if a cap is already detected with high confidence, skip the zoom pass.
3. Cap-region crop: for each bottle, crop the top r_{cap} portion with padding to include context.
4. Zoom and re-detect: resize the crop by factor α and run YOLO again to recover small caps.
5. Merge detections: combine pass-1 and pass-2 caps using IoU and keep the stronger box.

This strategy reduces false negatives for small caps while keeping the detector lightweight.

4.6 Complexity and runtime notes

Per-frame time complexity is dominated by model inference. Let $T_{det}^{(1)}$ be the full-frame detector latency, $T_{det}^{(2)}$ the zoomed cap pass, T_{cls} the cap classifier latency, and T_{fill} the water-surface detector latency. With N_b bottles in a frame, the worst-case latency is approximately

$$T_{frame} \approx T_{det}^{(1)} + N_b \cdot T_{det}^{(2)} + N_b \cdot T_{cls} + N_b \cdot T_{fill} + T_{post} \tag{4.3}$$

where T_{post} includes cropping, resizing, and result assembly. In practice N_b is small and the zoom pass is skipped when caps are already detected with high confidence.

Chapter 5

Results and Discussion

5.1 Overview

This chapter presents the implemented results obtained from the VisionFood QAI pipeline and discusses the practical behaviour observed during system validation. The evaluation focuses on the working inspection path for transparent beverage bottles, cap-quality verification, fill-level estimation, REMEDY routing, API integration, and database traceability.

The implemented system converts an input product image into a structured inspection result. The result contains detected defect evidence, confidence values, a final verdict, latency metadata, and visual overlays for operator review. This design makes the output useful not only for model evaluation but also for actual factory workflow, because each automated decision can be traced to a visible detection and a stored inspection record.

5.2 Experimental Setup

The project repository contains model weights, annotated inference outputs, dashboard source code, API routes, database repositories, and reporting utilities. The main evaluation context used in this phase is transparent-bottle inspection, where the system checks cap status and fill-level behaviour. Training and validation assets are stored under the `runs` directory, including the cap-inspection annotations, trained detector weights, inference predictions, and training metrics.

The cap detector training run was executed using the `a5000-balanced` training profile. The recorded hardware configuration includes an NVIDIA RTX A5000 GPU with approximately 24 GB of VRAM and 128 GB of system RAM. The training configuration used image size 736, batch size 16, 16 workers, RAM caching, automatic mixed precision, cosine learning-rate scheduling, multi-scale training, and mosaic closing near the end of training. The cap-related classes in the recorded training context are `defectCap`, `goodCap`, and `noCap`.

Table 5.1: Cap-inspection training context

Item	**Recorded value**
Product category	Beverage
Product sub-type	Transparent bottle
Inspection focus	Cap integrity
Model classes	`defectCap`, `goodCap`, `noCap`
Image size	736 pixels
Batch size	16
Training profile	`a5000-balanced`
Hardware	NVIDIA RTX A5000 GPU, approximately 24 GB VRAM

5.3 Model Training Results

The final recorded epoch of the cap-inspection detector training run shows strong localisation and classification performance for the bottle-cap task. The recorded precision was 0.8691, recall was 0.9729, mAP@0.5 was 0.9583, and mAP@0.5:0.95 was 0.7861. These values indicate that the model is especially strong at recovering relevant cap instances, which is important in a safety-critical inspection pipeline where missed defects can allow a non-compliant product to pass.

Table 5.2: Final recorded detector metrics for cap inspection

Metric	**Recorded value**	**Interpretation**
Precision	0.8691	Most positive detections correspond to a valid cap-status class.
Recall	0.9729	The model recovers a high proportion of relevant cap instances.
mAP@0.5	0.9583	Bounding-box detection is strong at the IoU 0.5 threshold.
mAP@0.5:0.95	0.7861	Localisation remains useful across stricter IoU thresholds.

The high recall is especially valuable for the implemented two-stage cap inspection logic. A missed cap defect has higher operational cost than a moderate number of false positives, because the REMEDY and review layers can still route uncertain products for review, while a missed defect may continue downstream. The model therefore supports the system objective of prioritising reliable defect capture over purely aesthetic prediction output.

5.4 Inspection Output Validation

The inference validation outputs stored in the repository show that the detector produces structured per-image records containing the source image path, product category, product sub-type, inspection focus, defect status, detection count, class name, confidence, bounding box, normalised bounding box, and area fraction. This confirms that the inspection pipeline produces both visual and machine-readable evidence. Such structured output is required for later aggregation in analytics, report generation, and traceable decision review.

The validation examples show three important cases:

- Defective caps are detected as `defectCap` with bounding boxes and confidence values.
- Correct caps are detected as `goodCap`, allowing the system to distinguish between actual defects and acceptable products.
- Missing-cap examples are represented through the `noCap` class, which enables direct routing to rejection or operator review depending on confidence and severity.

The fill-level path was validated through the algorithm described in Chapter 4. The water-surface detector localises the liquid-air interface, after which the geometric fill ratio is calculated using the corrected bottle top, bottle bottom, and detected surface coordinate. The output is converted into one of three verdicts: `underfill`, `normal`, or `overfill`. This separation between detection and measurement is useful because the surface detector supplies visual evidence, while the final fill decision is derived from a transparent equation.

5.5 System-Level Results

At the system level, the project demonstrates an end-to-end inspection workflow rather than an isolated model. A camera or uploaded image enters the API, the inference pipeline produces detections and a verdict, and the REMEDY engine scores severity. The FastAPI inspection route accepts base64-encoded JPEG or PNG images and returns a structured `InspectionResult`. The database layer stores inspection history, defect details, remediation actions, product information, and production run metadata.

The system supports the following output behaviours:

- Clean images produce a `PASS` verdict when no defect evidence is detected.
- High-confidence defects produce `FAIL` or escalation-oriented verdicts depending on uncertainty and threshold configuration.
- REMEDY maps common defect types to corrective actions such as `RELABEL`, `REFILL`, `REPACK`, `CLEAN`, or `REJECT`.
- Inspection history remains available for later review, analytics, and report generation.

This result is important because a manufacturing inspection system must do more than classify an image. It must decide what the line should do next, preserve evidence, and support accountability when an automated decision is questioned.

5.6 Discussion

The results show that VisionFood QAI is strongest when treated as an integrated quality-inspection workflow. The detector provides visual evidence, the fill-level logic provides measurable compliance checks, and the REMEDY engine turns defect evidence into action. This integration is the main achievement of the project.

The main limitation is that the available training evidence is stronger for bottle and cap inspection than for the full range of food and packaging categories described in the broader design. Transparent bottles, cap status, and fill level are currently the most mature inspection path. Additional data collection and validation will be required for flexible wrappers, rigid cans, rigid boxes, OCR date verification, and QR mismatch handling before the same confidence can be claimed across all product types.

Another limitation is deployment maturity. The repository includes API, dashboard, database, model, report, and REMEDY components, but real production deployment would require stable camera calibration, controlled lighting, production-line timing tests, and operator acceptance testing. These items are therefore carried into the future-work plan in Chapter 6.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

The VisionFood QAI project demonstrates an AI-assisted quality inspection system for food and beverage manufacturing. The implemented system combines computer vision, structured inspection output, uncertainty-aware decision logic, REMEDY-based corrective routing, API integration, persistent inspection records, and an operator-facing dashboard. The project therefore addresses the central problem identified in Chapter 1: manual inspection is difficult to scale, while simple rule-based inspection lacks flexibility for modern product variation.

The work confirms that transparent-bottle inspection can be handled through a practical combination of object detection, cap-quality classification, and geometric fill-level estimation. The cap-inspection training results show strong performance, with final recorded mAP@0.5 of 0.9583 and recall of 0.9729. The fill-level method provides explainable measurement because the final decision is derived from a visible liquid-surface detection and a defined fill-ratio equation. The REMEDY engine further improves practical usefulness by converting defect evidence into actions such as refill, relabel, clean, repack, or reject.

The user interface implementation is also a significant outcome. The dashboard provides manual inspection, live production run setup, inspection history, detail review, analytics, report download, and product registration. This means the project is not limited to model experimentation; it provides a working inspection environment that can be demonstrated, operated, and extended.

6.2 Key Learnings

The first key learning is that a quality-inspection system must be designed as a pipeline rather than as a single model. Detection accuracy matters, but the final value comes from evidence capture, decision logic, remediation, storage, and operator visibility. The second learning is that explainability can be built into system design through annotated images, explicit fill-ratio equations, confidence values, severity components, and traceable verdicts. The third learning is that product context is essential. SKU profiles, product sub-types, and container contents allow the same inspection framework to adapt to different production scenarios without changing the whole pipeline.

Another important learning is that user experience is part of the technical solution. An accurate model is difficult to trust if operators cannot understand what it detected or why it routed a product to a station. The dashboard therefore plays a direct role in reliability by making inspection evidence visible and actionable.

6.3 Comparative Analysis as Future Work

A complete comparative analysis is planned as future work because the current phase focuses on implementation and internal validation. The future comparative study should evaluate VisionFood QAI against three baselines:

- Manual visual inspection by trained operators.
- Classical rule-based machine vision using fixed thresholds and contours.
- A model-only detector without REMEDY routing, uncertainty handling, or dashboard traceability.

The comparison should measure detection accuracy, false rejection rate, false acceptance rate, average inspection latency, operator intervention rate, and remediation success rate. The expected benefit of VisionFood QAI is not only better detection, but also better operational handling of defects through severity scoring, evidence storage, and corrective routing.

6.4 Optimisation Roadmap

Optimisation is also reserved for future work because production optimisation should be based on larger deployment measurements rather than only laboratory examples. The first optimisation direction is model optimisation. The detector and classifier should be exported to ONNX, profiled on the target edge device, quantised where accuracy allows, and benchmarked under production image sizes. The second direction is pipeline optimisation. Detection, QR decoding, fill-level estimation, and result persistence can be parallelised where dependencies allow, while expensive modules such as OCR should remain SKU-gated and run only when required.

The third optimisation direction is camera and pre-processing optimisation. Stable lighting, fixed camera distance, automatic exposure control, and frame selection can reduce false positives and improve detection consistency.

6.4.1 User Interface Optimisation

User interface optimisation remains future work. The dashboard should be evaluated with operators and refined for faster review, clearer defect explanations, improved mobile responsiveness, better filtering in inspection history, and lower API load when viewing annotated media repeatedly.

6.5 Future Work

The following future extensions are recommended:

- Expand the dataset for flexible wrappers, rigid cans, rigid boxes, transparent bottles, label defects, contamination, and packaging damage.
- Add complete QR and OCR verification into the production inference path so barcode mismatch and printed-date mismatch can be handled as first-class quality defects.

- Validate the system on live conveyor-belt video with controlled lighting, realistic belt speed, and multiple product batches.
- Improve fill-level detection for difficult cases such as frosted bottles, curved bottles, reflective labels, and low-contrast liquids.
- Add edge deployment benchmarks on CPU-only and GPU-enabled devices to confirm latency under real production constraints.
- Extend REMEDY with feedback learning so repeated operator overrides can improve severity thresholds and routing rules.
- Optimise the user interface based on operator feedback, including inspection-history filtering, annotated-image loading, and clearer remediation prompts.
- Add role-based access control for operator, supervisor, and administrator workflows in the dashboard.
- Integrate generated daily and weekly reports with automated email or cloud storage delivery.

Future work should also include a user-interface validation study. Operators should be asked to use the dashboard on inspection examples and rate clarity, speed, confidence, and ease of corrective action.

6.6 Sustainable Development Goals Alignment

The project aligns with the Sustainable Development Goals (SDGs) shown in Table 6.1. The alignment is based on the system's contribution to industrial automation, waste reduction, and food safety.

6.7 Closing Summary

VisionFood QAI provides a foundation for practical AI-based quality inspection in food and beverage manufacturing. The system shows how detection, measurement, severity scoring, remediation, traceability, and user interface design can work together as one workflow. With further dataset expansion, production benchmarking, comparative evaluation, and deployment optimisation, the project can evolve from a capstone prototype into a stronger smart-factory inspection platform.

Table 6.1: SDG alignment for VisionFood QAI

SDG	Justification	Strength
SDG 9: Industry, Innovation and Infrastructure	The system contributes through AI-driven automation in manufacturing. It uses computer vision and edge deployment to enable smart factory operations. This improves inspection speed, accuracy, and scalability, leading to increased industrial productivity and technological advancement.	High (3)
SDG 12: Responsible Consumption and Production	The REMEDY engine reduces waste by identifying defects that can be corrected instead of rejected. By enabling actions like relabeling, refilling, and cleaning, the system minimises material wastage and promotes efficient resource utilisation in production processes.	Medium (2)
SDG 3: Good Health and Well-being	The system detects contamination and packaging defects that may affect food safety. By ensuring only safe and compliant products reach consumers, it contributes to improved public health and reduces risks associated with unsafe food products.	Medium (2)

Appendices

6.8 Dashboard Screenshots

This section includes representative screenshots from the operator-facing dashboard. These figures document the working frontend views used for real-time monitoring and manual model inspection.

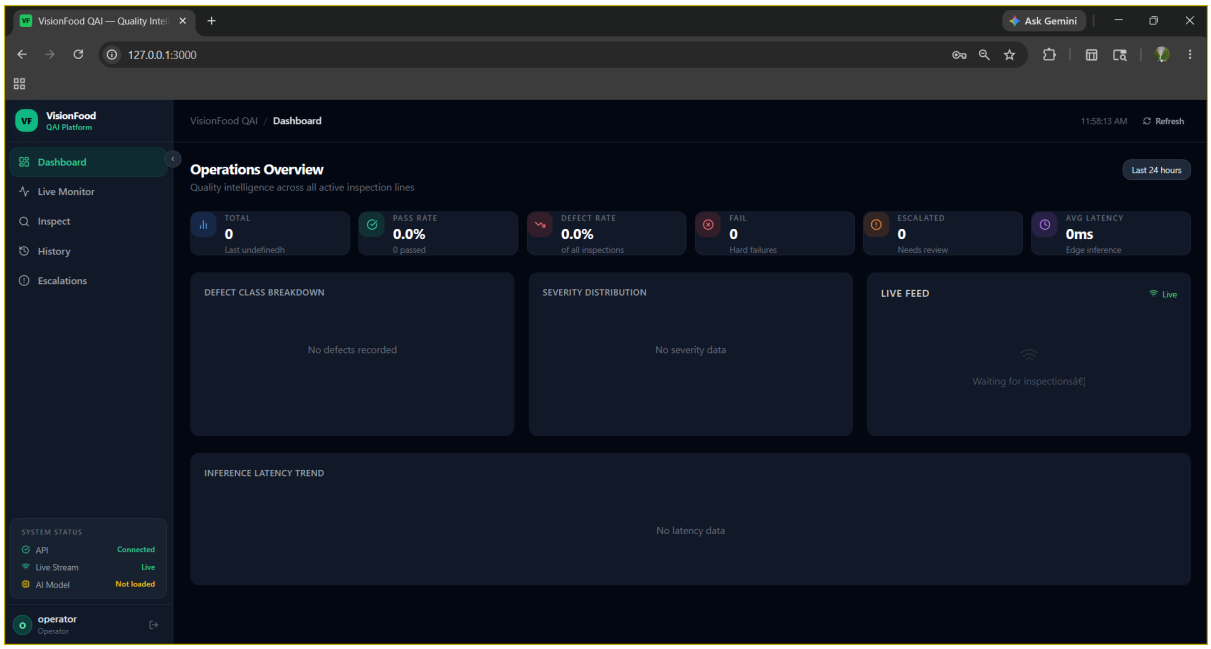


Figure 6.1: VisionFood QAI dashboard home page

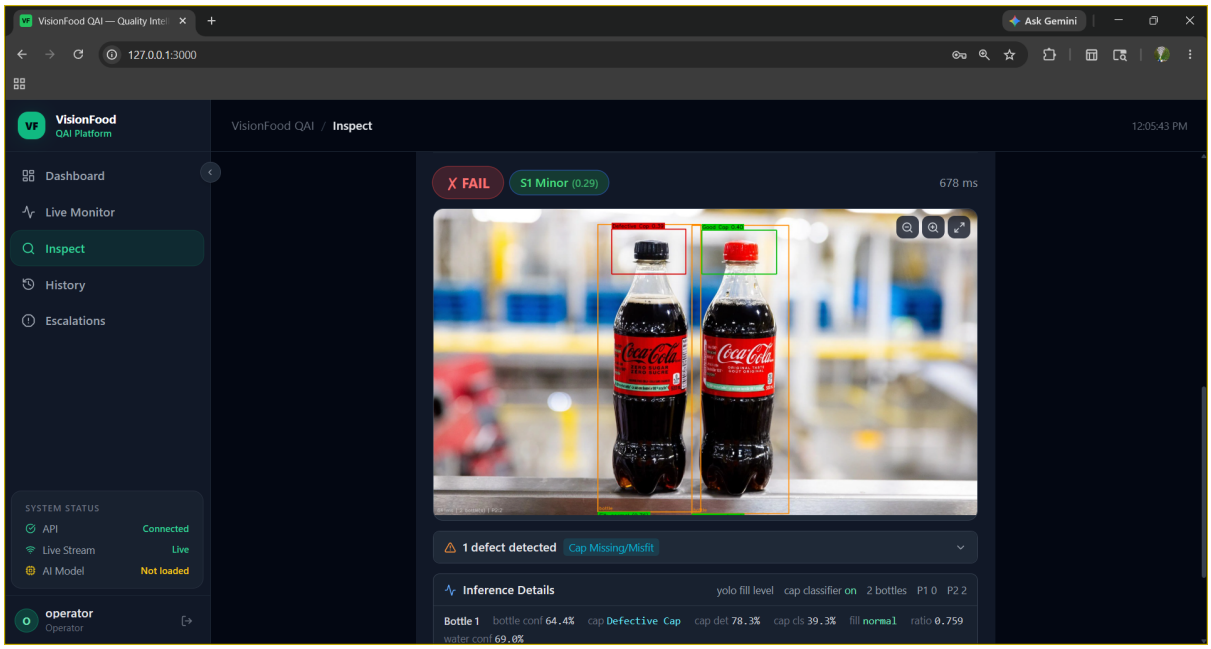


Figure 6.2: Manual inspection interface with uploaded product image and inference controls

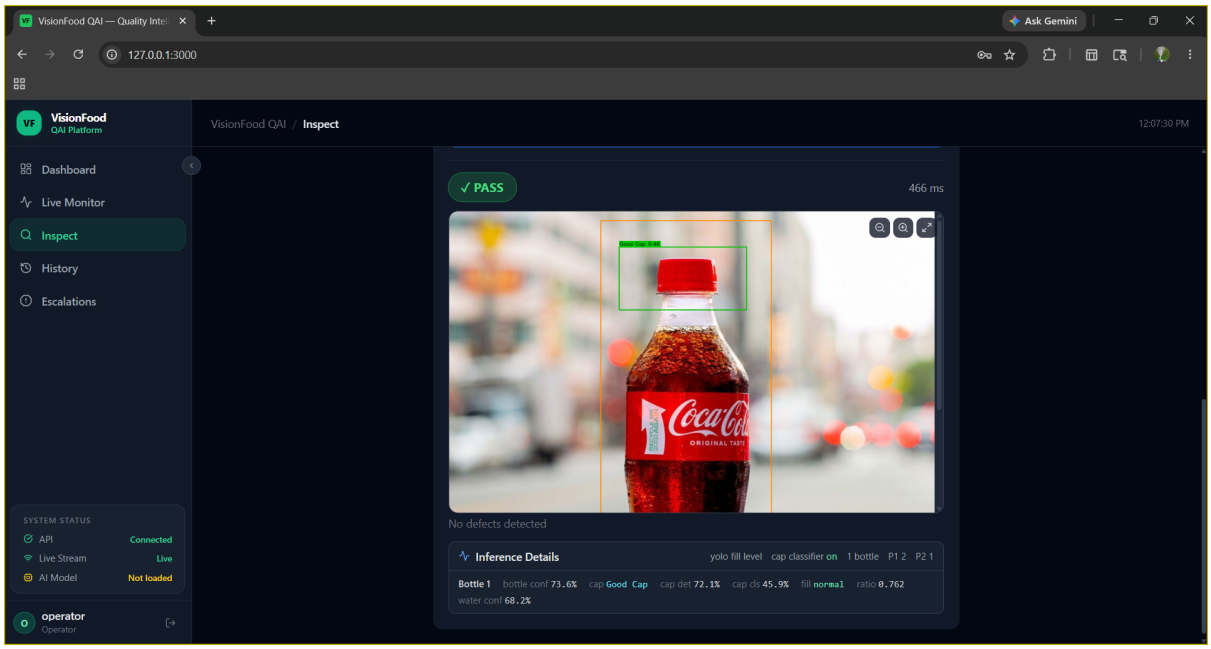


Figure 6.3: Manual inspection result view showing annotated output and inference details

6.9 Code Samples

This section records representative code fragments from the inference layer used in the implementation. The first sample is taken from `inference/bottle_cap_v3/detect_v3.py` and shows the bottle-guided crop used to focus cap detection on the top region of the bottle.

```
def _get_cap_crop_region(self, bottle_bbox, img_h, img_w):
    """Get the top region of a bottle where the cap lives."""
    x1, y1, x2, y2 = bottle_bbox
    bw, bh = x2 - x1, y2 - y1

    cap_h = int(bh * self.cfg.cap_region_ratio)
    pad_w = int(bw * self.cfg.crop_pad)
    pad_h = int(cap_h * self.cfg.crop_pad)

    cx1 = max(0, x1 - pad_w)
    cy1 = max(0, y1 - pad_h)
    cx2 = min(img_w, x2 + pad_w)
    cy2 = min(img_h, y1 + cap_h + pad_h)
    return cx1, cy1, cx2, cy2

def _zoom_crop(self, crop: np.ndarray) -> np.ndarray:
    """Upscale crop to improve small-object detection."""
    h, w = crop.shape[:2]
    scale = self.cfg.zoom_scale
    new_w, new_h = int(w * scale), int(h * scale)

    max_px = self.cfg.max_zoom_px
    if max(new_w, new_h) > max_px:
        ratio = max_px / max(new_w, new_h)
        new_w, new_h = int(new_w * ratio), int(new_h * ratio)

    if new_w < self.cfg.min_crop_px or new_h <
       self.cfg.min_crop_px:
        return None
    return cv2.resize(crop, (new_w, new_h),
       interpolation=cv2.INTER_LINEAR)
```

Listing 6.1: Cap-region cropping and zooming for cap detection

```
cx1, cy1, cx2, cy2 = self._get_cap_crop_region(
    bottle["bbox"], h, w
)
```

```
crop = img[cy1:cy2, cx1:cx2]
if crop.size == 0:
    continue

zoomed = self._zoom_crop(crop)
if zoomed is None:
    continue

r2 = self.model.predict(
    zoomed,
    conf=cfg.cap_conf,
    iou=cfg.iou_thresh,
    device=cfg.device,
    verbose=False
)
```

Listing 6.2: Applying the cap crop during the second detection pass

The fill-level calculation is taken from `inference/yoloWithFillLevel.py`. It trims the bottle bounding box, clamps the detected water-surface location to the valid bottle area, and converts the vertical distance from the corrected bottle bottom to the water surface into a normalized fill ratio.

$$\text{fill ratio} = \frac{y_{\text{bottom,corr}} - y_{\text{water}}}{y_{\text{bottom,corr}} - y_{\text{top,corr}}}$$

```
def compute_fill_ratio(bottle_bbox: list[int],
                       water_bbox: list[int]) -> float:
    bx1, by1, bx2, by2 = bottle_bbox
    bottle_h = by2 - by1

    by1_corr = by1 + int(bottle_h * BOTTLE_TOP_TRIM)
    by2_corr = by2 - int(bottle_h * BOTTLE_BOT_TRIM)
    bottle_h_corr = max(by2_corr - by1_corr, 1)

    water_y = (water_bbox[1] + water_bbox[3]) // 2
    water_y = max(by1_corr, min(by2_corr, water_y))

    fill_from_bottom = by2_corr - water_y
    fill_ratio = fill_from_bottom / bottle_h_corr
    return round(fill_ratio, 3)

def fill_verdict(fill_ratio: float,
```

```
                    underfill_thresh: float,
                    overfill_thresh: float) -> str:
    if fill_ratio < underfill_thresh:
        return "underfill"
    elif fill_ratio > overfill_thresh:
        return "overfill"
    else:
        return "normal"
```

Listing 6.3: Mathematical fill-ratio computation and verdict

```
best_water = None
best_water_conf = 0.0

for box in r_fill[0].boxes:
    wx1, wy1, wx2, wy2 = [int(v) for v in box.xyxy[0].tolist()]
    water_cx = (wx1 + wx2) / 2

    if bx1 <= water_cx <= bx2 and by1 <= wy1 <= by2:
        conf = float(box.conf[0])
        if conf > best_water_conf:
            best_water_conf = conf
            best_water = [wx1, wy1, wx2, wy2]

if best_water:
    fill_ratio = compute_fill_ratio(bottle["bbox"], best_water)
    fill_level = fill_verdict(
        fill_ratio,
        underfill_thresh,
        overfill_thresh
    )
```

Listing 6.4: Selecting the water-surface box for the detected bottle

6.10 Detailed Matrices

Table 6.2: Implementation Traceability Matrix

Implementation Area	Source File	Traceable Purpose
Cap-region crop and zoom	`inference/bottle_cap_v3/detect_v3.py`	Uses the detected bottle box to crop the upper bottle region, enlarge the crop, and improve cap detection for small cap objects.
Fill-level computation	`inference/yoloWithFillLevel.py`	Converts bottle and water-surface detections into a normalized fill ratio and classifies underfill, normal fill, or overfill.
Inspection orchestration	`inference/yoloWithFillLevel.py`	Combines bottle, cap, label, and fill-level detections into one inspection decision for each product instance.
User interface workflow	Dashboard implementation files	Presents inspection outcomes, summaries, and review-oriented controls discussed in Chapter 4.

Table 6.3: Cap Detection Crop Matrix

Parameter	Role	Effect on Inspection
`cap_region_ratio`	Selects the top portion of the bottle bounding box.	Keeps the second-pass detector focused on the expected cap location.
`crop_pad`	Adds horizontal and vertical context around the cap crop.	Reduces missed caps caused by slight bottle-box or cap-position variation.
`zoom_scale`	Enlarges the crop before the second detection pass.	Improves visibility of small caps in the detector input.
`max_zoom_px` and `min_crop_px`	Bound the resized crop dimensions.	Prevents extremely large crops and rejects crops too small for reliable detection.

Table 6.4: Fill-Level Decision Matrix

Condition	Verdict	Interpretation
`fill_ratio < underfill_thresh`	Underfill	The detected water surface is too low relative to the corrected bottle height.
`underfill_thresh <= fill_ratio <= overfill_thresh`	Normal	The product fill level lies inside the accepted operating range.
`fill_ratio > overfill_thresh`	Overfill	The detected water surface is too high relative to the corrected bottle height.
No valid water-surface box	Review required	The fill model did not provide a water-surface box that could be associated with the bottle.

References

[1] O. Farhangi, E. Sheidaee, and A. Kisalaei, "Machine vision for detecting defects in liquid bottles: An industrial application for food and packaging sector," *Cluster Computing and Data Science*, 2024.

[2] N. Banús, I. Boada, P. Xiberta, P. Toldrà, and N. Bustins, "Deep learning for the quality control of thermoforming food packages," *Scientific Reports*, vol. 11, 2021. DOI: https://doi.org/10.1038/s41598-021-01254-x.

[3] R. R. Subramanian, M. Dhamini, M. Srija, M. Vaishnavi, and M. Vidyadhari, "Revolutionizing manufacturing quality control with optimized resnet50: A deep learning approach for real-time defect detection," in *2025 International Conference on Computational Robotics, Testing and Engineering Evaluation (ICCRTEE)*, 2025.

[4] K. K. S, M. C. G, and R. T. V, "Deep learning based product defect detection for sustainable smart manufacturing," in *2024 5th IEEE Global Conference for Advancement in Technology (GCAT)*, 2024, pp. 1–6. DOI: https://doi.org/10.1109/GCAT62922.2024.10924069.

[5] K. G. Liakos, V. Athanasiadis, E. Bozinou, and S. I. Lalas, "Machine learning for quality control in the food industry: A review," *Foods*, vol. 14, 2025.

[6] L. Xu, M. Chen, and H. Zhu, "Multiple-instance learning for surface defect detection using weak supervision and image-level labels," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 3, pp. 3456–3468, 2024. DOI: https://doi.org/10.1109/TII.2023.3321456.

[7] B. Chatchapol, W. Autanan, and W. Anan, "Liquid filling quality control system for beverage industry using image processing and cnn," in *2024 24th International Conference on Control, Automation and Systems (ICCAS)*, 2024, pp. 1–6.

[8] S. Sanjay et al., "Food oil quality and usage detection using ai," in *2024 10th International Conference on Communication and Signal Processing (ICCSP)*, 2024, pp. 1190–1194. DOI: https://doi.org/10.1109/ICCSP60870.2024.10543730.

[9] F. Xiong, N. Kühl, and M. Stauder, "Designing a computer-vision-based artifact for automated quality control: A case study in the food industry," *Annals of Operations Research*, 2024. DOI: https://doi.org/10.1007/s10696-023-09523-9.

[10] K. K. S and M. C. G, "Automated product defect detection using image processing techniques for effective sorting and quality assurance: A survey," *International Journal of Innovative Science and Research Technology (IJISRT)*, vol. 9, no. 6, 2024.

[11] X.-T. Nguyen, T.-T. Mac, Q.-D. Nguyen, and H.-A. Bui, "An industrial system for inspecting product quality based on machine vision and deep learning," *Vietnam Journal of Computer Science*, vol. 12, no. 2, pp. 193–208, 2025. DOI: https://doi.org/10.1142/S2196888825400032.

[12] G. Liu, S. Zhang, L. Wang, X. Li, and G. Li, "Research on mechanical automatic food packaging defect detection model based on improved yolov5 algorithm," *PLOS ONE*, vol. 20, 2025. DOI: https://doi.org/10.1371/journal.pone.0321971.

[13] N. C. Gediya, "Smart manufacturing: Real-time quality control with ai and image recognition," *Conference Proceedings*, 2024.

[14] P. Krishnan, U. S. Ebenezar, R. Ranitha, N. Purushotham, and T. S. Balakrishnan, "Ai-driven intelligent iot systems for real-time food quality monitoring and analysis," in *2024 International Conference on Trends in Quantum Computing and Emerging Business Technologies*, 2024, pp. 1–5. DOI: https://doi.org/10.1109/TQCEBT59414.2024.10545305.

[15] T. Du, "Online measurement and detection technology and its optimized application in intelligent manufacturing environment," *Procedia Computer Science*, vol. 262, pp. 83–91, 2025.

[16] Z. Cao, Z. Wu, and J. Gray, "Rfid tag-integrated multi-sensors with aiot cloud platform for food quality analysis," *Electronics*, vol. 15, no. 1, 2025.

[17] A. M. Truong and H. Q. Luong, "A non-destructive, autoencoder-based approach to detecting defects and contamination in reusable food packaging," *Current Research in Food Science*, vol. 8, p. 100 758, 2024.

[18] A. G. Ribeiro, L. Vilaça, C. Costa, T. S. da Costa, and P. M. Carvalho, "Automatic visual inspection for industrial application," *Journal of Imaging*, vol. 11, 2025.